Wilfrid Laurier University

CP351 Final Project

Exploring QAOA on PennyLane and Qiskit for Portfolio Optimization

Prepared by

Anton Nemchinski

Professor

Shohini Ghose

April 17, 2026

1 Introduction & Motivation

Combinatorial optimization problems are one of the most ubiquitous mathematical problems that we encounter in our day-to-day lives, from planning the shortest path from one lecture to the next, or choosing the best restaurant while considering the conflicting tastes of your party. Furthermore, the ability to solve for the best state given a set of constraints has many applications in industry and academia, from reducing traffic, optimizing a supply chain, or even training a neural network. Fundamentally, what makes all these problems difficult is combinatorial explosion, the fact that the number of possible states scales exponentially with problem size, which makes exhaustive brute-force searches intractable.

Many quantum algorithms have properties that are very attractive to optimization problems, since they leverage superposition and entanglement in clever ways to mitigate complexity scaling. Taking factoring as an example, the time complexity of the best-known classical algorithm scales exponentially with problem size, while Shor's algorithm uses a clever application of the Quantum Fourier Transform to reduce time complexity to polynomial time. While the Quantum Approximate Optimization Algorithm (QAOA) explored in this report does not purport to lead to similar dramatic reductions, it is at the forefront of quantum computing research because it is not only theoretically possible, but it is well-suited for existing noisy and imperfect quantum computers.

In this report, we focus on applying the QAOA to financial portfolio optimization, specifically to mean-variance analysis which aims to find the optimal portfolio that simultaneously maximizes expected returns while minimizing risk. We start by defining the problem, encoding it in a QAOA-friendly way and explaining how QAOA solves it, followed by a test on a simulator in Pennylane, and a run on real quantum hardware with IBM Qiskit.

2 The Mean-Variance Binary Optimization Problem

The mean-variance portfolio optimization problem studied in this report is

$$\min_{x \in \{0,1\}^n} \quad q x^T \Sigma x - \mu^T x \quad \text{subject to} \quad 1^T x = B \tag{1}$$

where the goal is to identify the optimal binary decision vector x. Each entry x_i indicates whether asset i is selected, so $x_i = 1$ means the asset is included in the portfolio and $x_i = 0$ means it is excluded. This binary representation is especially convenient for QAOA because each portfolio can be read directly as a bitstring. The vector μ represents the expected returns of the assets, the covariance matrix Σ measures how the asset returns vary together, and thus the term $x^T \Sigma x$ represents portfolio risk (since risk increases with correlation between assets), while the parameter q controls how strongly risk is weighted relative to return. The constraint $1^T x = B$ enforces the budget rule that exactly B assets must be selected.

For the small examples in this project, the optimal solution can be found by brute-force enumeration, checking every bit string with B 1's and comparing by objective value.

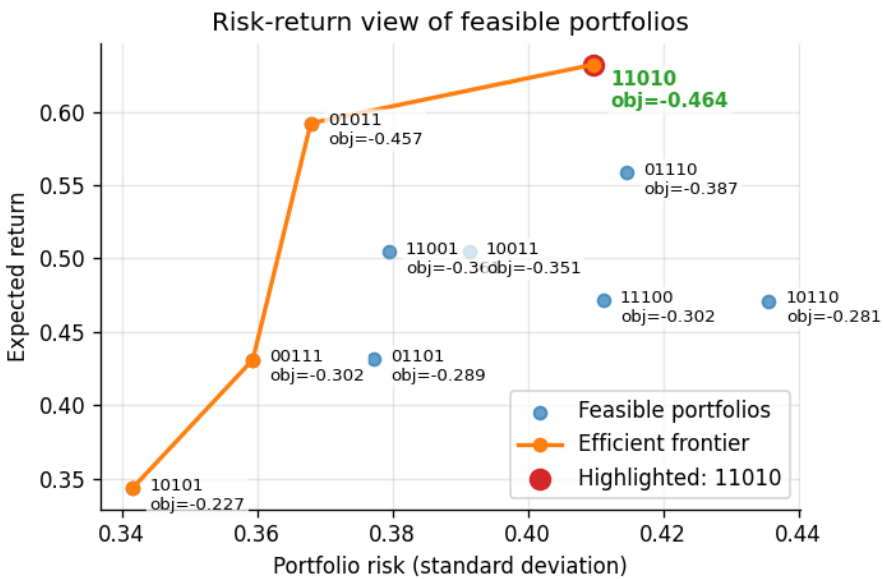

Figure 1: Risk-return view of all feasible portfolios for a 5-asset $B = 2$ example. Each point represents a feasible portfolio, with risk measured by portfolio standard deviation and return measured by expected return. Portfolios on the *efficient frontier* are not dominated by any other feasible portfolio, so no other choice offers both higher return and lower risk at the same time. Portfolios below this boundary are dominated and therefore suboptimal. The highlighted single portfolio is chosen once the risk-aversion parameter q is fixed.

3 From Portfolio Optimization to a QAOA Circuit

The Quantum Approximate Optimization Algorithm (QAOA) is a hybrid quantum-classical algorithm for approximate combinatorial optimization. At its core are two *Hamiltonians*, a type of operator that assigns an energy value to each computational basis state. For optimization problems, this is useful because the classical objective can be encoded into a *cost Hamiltonian* whose low-energy states correspond to good portfolios. Therefore, the first task is to reformulate the constrained binary mean-variance objective into such a Hamiltonian.

The first step is to rewrite it as a Quadratic Unconstrained Binary Objective (QUBO) as follows:

$$qx^T\Sigma x - \mu^T x + \lambda(1^T x - B)^2 \tag{2}$$

The penalty term $\lambda(1^T x - B)^2$ absorbs the budget constraint directly into the objective, so the algorithm can operate upon one unconstrained binary objective function. The next step is to map the binary variables into qubit operators using:

$$x_i = \frac{1 - Z_i}{2}, \tag{3}$$

where Z_i is the Pauli-Z operator acting on qubit i. After this substitution, the objective becomes a cost Hamiltonian built from a constant term, single-qubit Z_i terms, and pairwise $Z_i Z_j$ terms. The second Hamiltonian used by QAOA is the *mixer Hamiltonian.* In this project, the standard X-mixer was used,

$$H_M = \sum_i X_i, \tag{4}$$

because it is simple, standard, and easy to interpret: it gives the circuit a way to flip between different binary include/exclude asset choices, rather than staying locked into one portfolio.

QAOA then builds a quantum circuit from these two Hamiltonians. This *parameterized* trial circuit is called an *ansatz.* The circuit begins in a uniform superposition, usually prepared with Hadamard gates, so that all candidate bitstrings start on equal footing. It then alternates between the application of a cost unitary,

$$U_C(\gamma) = e^{-i\gamma H_C}, \tag{5}$$

and a mixer unitary,

$$U_M(\beta) = e^{-i\beta H_M}. \tag{6}$$

A single QAOA layer consists of one application of each, and repeating this p times gives a depth-p circuit. The γ parameters control how strongly the cost Hamiltonian affects the state, while the β parameters control how strongly the mixer redistributes amplitude through the search space. Theoretically, increasing p makes the ansatz more expressive, although deeper circuits are also harder to optimize and harder to run reliably on hardware.

The classical optimizer then works to optimize the parameters of the ansatz. It tunes (γ, β) using traditional optimizers like ADAM to minimize the expected value of the cost Hamiltonian:

$$\langle \psi(\gamma, \beta) | H_C | \psi(\gamma, \beta) \rangle. \tag{7}$$

Minimizing this expected objective improves the final quantum state because it pushes the circuit toward a probability distribution with more weight on low-cost portfolios and less weight on poor or infeasible ones. Here, $\langle \psi(\gamma, \beta)$ is the QAOA *ansatz state*, meaning the quantum state produced by the parameterized circuit after applying the cost and mixer layers with the current angles γ and β. Equation 7 represents the *expectation value* of the cost Hamiltonian H_C, or the average objective value of the output distribution produced by the current ansatz. The classical optimizer improves the state over many optimization steps by adjusting (γ, β) to lower this average cost. Once the parameters are optimized, the final state is measured repeatedly using a fixed number of *shots*. Each shot produces one bitstring, or one candidate portfolio, and the collection of many shots gives a sampled distribution over portfolios. In practice, the final answer is then taken from this distribution of bitstrings, either as the best feasible sampled portfolio or the most probable good portfolio. QAOA is especially attractive for existing Noisy Intermediate-Scale Quantum (NISQ) hardware because it uses a relatively shallow parameterized circuit and only aims for approximate solutions.

4 QAOA Simulation with Pennylane

PennyLane was used as the main prototyping environment for this project because it made the full QAOA workflow straightforward to implement and test. The notebook first generates a random portfolio instance, computes the exact classical optimum by brute-force, and then derives the corresponding QUBO and cost Hamiltonian. From there, a QAOA circuit was built using the standard X-mixer and optimized on an ideal (noise-free) simulator with ADAM as the classical optimizer. This gave a clean end-to-end pipeline: define the finance problem, translate it into a Hamiltonian, optimize the ansatz, and then compare the sampled portfolios against the true classical benchmark.

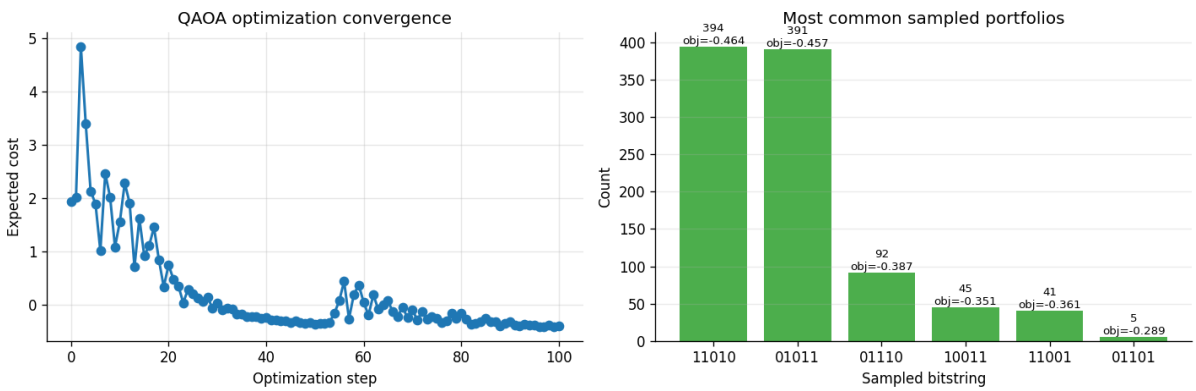

Figure 2: PennyLane results for the 5-asset instance from Figure 1. The optimization convergence plot shows the expected cost decreasing during training, while the sampled bitstring histogram shows that the final distribution is concentrated on the strongest portfolios. Above each bar is the objective value and the number of times it was sampled.

Figure 2 shows a 5-asset run under ideal conditions, with 100 optimization steps, an ansatz depth of 30, and 1000 shots. The convergence curve on the left shows that the expected cost decreased steadily during optimization, demonstrating correct performance by the optimizer. The histogram on the right shows that the final sampled distribution was concentrated almost entirely on the two strongest feasible portfolios, with the largest probabilities assigned to the portfolios with the strongest classical objective values (indicated above each portfolio's bar, the lower the better). Specifically, the two most common sampled portfolios were `11010` and `01011`, with objectives -0.464 and -0.457, respectively. This represents an ideal result, as the algorithm correctly modelled the relative strength of candidate portfolios, and by the final stage infeasible portfolios were

completely filtered out.

After verifying the ideal case, the same workflow and example was extended to a noisy simulator in order to identify configurations that were more likely to remain useful on real hardware. Noise was modelled using PennyLane's mixed-state simulator through a layer-level depolarizing channel applied after each cost and mixer layer, which essentially randomly perturbs each qubit. This is not really an accurate model of the noise that will be encountered on IBM hardware, but it was useful to test the general robustness of the implementation. The most interesting result of this test is shown in Figure 3. Unlike the ideal scenario where depth p could be arbitrarily increased for better performance, increasing depth beyond $p = 3$ led to decreased performance. For both 3-asset and 4-asset cases, moderate depth around $p = 3$ gave the best overall tradeoff, while deeper circuits lost both optimum probability and feasibility under noise. This was one of the most important practical findings in the PennyLane study, because it showed that the best ideal configuration is not automatically the best hardware-facing configuration.

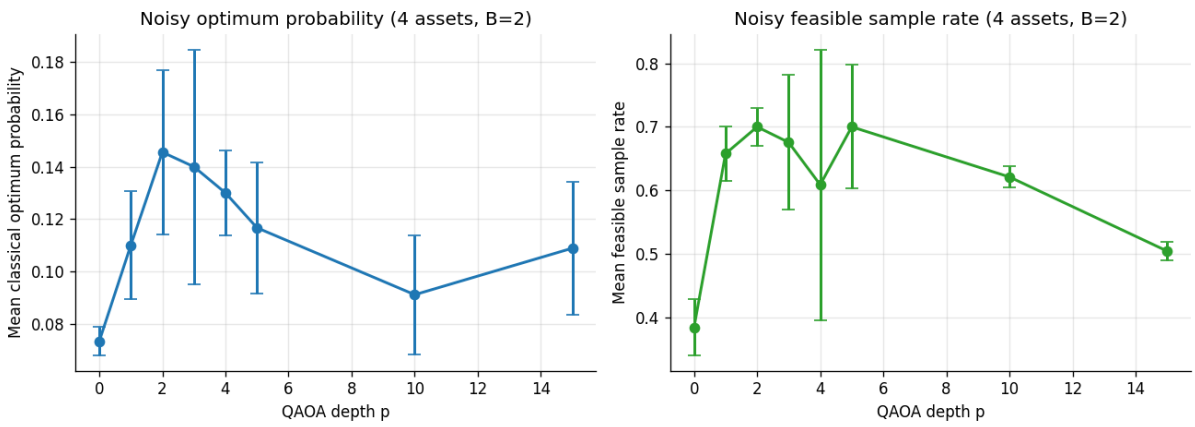

Figure 3: Noisy PennyLane depth study for the 4-asset, budget-2 case. The left panel plots the mean probability of sampling the classical optimum as a function of QAOA depth p, while the right panel plots the mean feasible sample rate. Each point is the average over three seeds, and the error bars show one standard deviation across those runs.

Overall, the PennyLane tests established two main points. Firstly, in an ideal simulation, the QAOA successfully placed highest probability on the highest quality feasible portfolios. Additionally, once noise was introduced, increasing the depth of the circuit no longer led to better performance.

5 QAOA on IBM Hardware: Transfer Challenges and Results

The IBM implementation reused the same finance model, QUBO formulation, and cost-Hamiltonian derivation as the PennyLane workflow, but changed the execution stack. The quantum circuit construction, sampling, and hardware execution were implemented in Qiskit. To keep hardware runs practical and within the IBM open plan budget, the IBM tests switched to a 3-asset example rather than the up to 5-asset instances used in PennyLane. This reduces the cost of transpilation, sampling, and also underscores the limitations of current quantum hardware. Figure 4 shows the feasible portfolios for this 3-asset instance. The highlighted portfolio `110` is the classical optimum under the objective, with two other feasible portfolios existing, and 5 additional infeasible portfolios (not shown here).

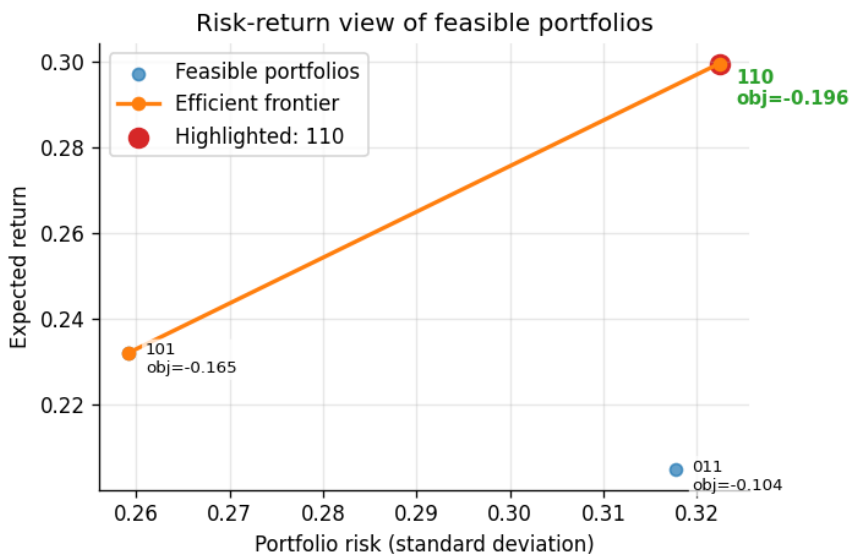

Figure 4: Risk-return view of the feasible portfolios for the 3-asset instance used in the IBM study. The highlighted portfolio `110` is the classical optimum under the chosen objective.

While running the full workflow on real hardware was possible for the 3-asset example, many hyperparameters had to be tuned to reduce computational costs. Specifically, the step count was reduced to 70 which reduces the length of time for the optimizer to minimize the objective, and shots were reduced to 50 which makes small differences in

the final distribution difficult to distinguish. Additional testing confirmed the results of the noisy PennyLane simulations, increasing the depth to $p = 4$ made the results substantially worse and optimization incredibly noisy. The circuit was also struggling to consistently filter out infeasible portfolios until the penalty λ was increased, but this came at the cost of greater volatility in the optimizer loop as seen in Figure 5.

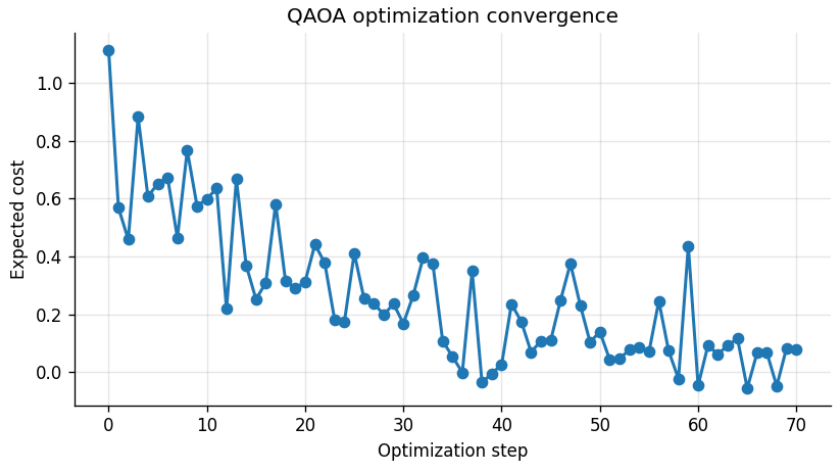

Figure 5: The optimizer trajectory for the tuned IBM hardware run on the 3-asset problem. The expected cost does decrease with increased steps, but it is significantly more volatile than all previous simulations.

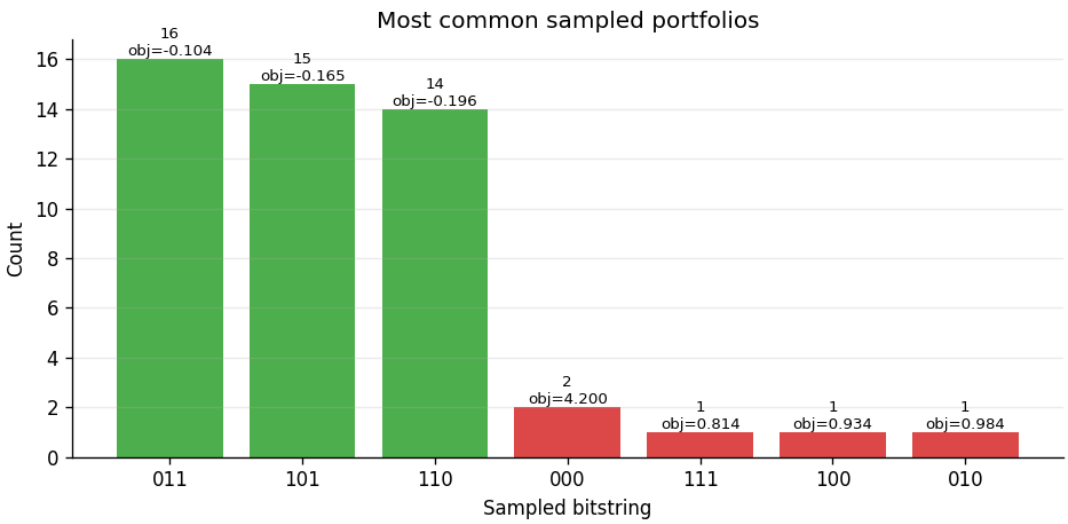

Figure 6: Results for IBM hardware run for the 3-asset problem. Feasible portfolios are marked in green while infeasible are marked in red.

The final 3-asset results in Figure 6 illustrate these tradeoffs: although feasible portfolios still dominate the sampled distribution, the worst feasible portfolio was sample the most often, and some infeasible states remain visible. Additionally, while these are

real QAOA results on quantum hardware, significant tuning was done to get this single randomly generated example to give the desired results.

5.1 Examining the Effects of Transpilation

Execution on real hardware introduces another non-obvious layer of complexity. Despite the logical circuit relying only on three qubits to represent the optimized portfolio, this circuit must be translated to match the hardware backend before execution. This is called *transpilation* where logical level ideal operations such as cost-evolution and mixer layers are mapped to the limited native gate set of the IBM backend. For the 3-asset problem, transpilation statistics were logged for both an *initial* and a *final* circuit because the optimization loop begins with one set of QAOA parameters and ends with another. The overall ansatz structure is the same in both cases, but since the actual gate rotations inside it are different the transpiler may simplify the circuit slightly differently at each execution step.

For the above 3-asset hardware run, the transpilation results were very similar before and after optimization. The initial circuit transpiled to depth 114, size 173, and 30 CZ gates, while the final circuit transpiled to depth 115, size 172, and again 30 CZ gates. Here, the *depth* measures how many sequential layers of operations the hardware must execute, while the *size* counts the total number of gates in the compiled circuit. In addition to the CZ gates, the transpiled circuit also contained single-qubit operations such as R_z and $\sqrt{X}$, but the CZ count is the most important diagnostic because it measures how many entangling two-qubit operations the hardware must perform, and these are typically the most noise-sensitive part of the circuit. The near-identical initial and final statistics show that the optimizer mainly changed the gate angles rather than the compiled circuit structure itself. However, the main takeaway is that even though the logical problem involved only 3 qubits and a depth of 3, the hardware-executable circuit was still fairly deep and contained a substantial number of entangling gates.

6 Conclusion

This project showed that applying QAOA to real simple optimization problem is workable, and there are many opportunities to experiment and optimize quantum algorithms on modern simulators.

In the ideal PennyLane simulation, increasing circuit depth reliably improved performance because the ansatz became more expressive and was better able to place probability mass on strong feasible portfolios. However, this trend did not persist once noise and hardware constraints were introduced. In the noisy simulator, performance improved only up to moderate depth before beginning to degrade, and the IBM hardware results showed even more clearly that deeper circuits were harder to execute reliably. This pattern underscores the main challenge of applying QAOA in the NISQ era: while deeper circuits can represent better solutions to more complex problems, hardware noise, finite-shot estimation, and transpilation overhead can erase those gains.

The IBM implementation also highlighted how expensive and fragile real-device experimentation can be. Even for a tiny 3-asset problem, the transpiled circuit became much deeper than its logical form suggested, and full hardware optimization was far more sensitive to hyperparameters than local simulation. This was a useful reminder that current hardware can execute small variational circuits, but repeated optimization directly on the device is still a much more difficult task than validating a final optimized circuit.

However, finance is only one possible application area for QAOA, and there is substantial ongoing research aimed at improving its performance and scalability for practical optimization problems. Current experimental work suggests that increasing QAOA depth can improve solution quality on real hardware. In particular, the PostQuantum article (7) highlights Google's 2021 hardware study (4), which reported improved performance as the depth increased from $p = 1$ to $p = 3$ on certain optimization instances with up to 23 qubits. Overall, QAOA remains a promising near-term approach for optimization on realistic quantum hardware, although its practical effectiveness still depends strongly on the application and the limitations of the device.

References

[1] Assel, B. (2025), 'Where are we with shor's algorithm?'.

URL: *https://towardsdatascience.com/where-are-we-with-shors-algorithm/*

[2] Chen, J. (2025), 'Mean-variance analysis: Definition, example, and calculation'.

URL: *https://www.investopedia.com/terms/m/meanvariance-analysis.asp*

[3] *Finance Tutorials* (2026).

URL: *https://qiskit-community.github.io/qiskit-finance/tutorials/index.html*

[4] Harrigan, M. P., Sung, K. J., Neeley, M., Satzinger, K. J., Arute, F., Arya, K., Atalaya, J., Bardin, J. C., Barends, R., Boixo, S. et al. (2021), 'Quantum approximate optimization of non-planar graph problems on a planar superconducting processor', *Nature Physics* **17**, 332–336.

URL: *https://arxiv.org/abs/2004.04197*

[5] Haugh, M. (2016), 'Mean-variance optimization and the CAPM', IEOR E4706: Foundations of Financial Engineering lecture notes, Columbia University.

URL: *https://www.columbia.edu/ mh2078/FoundationsFE/MeanVariance-CAPM.pdf*

[6] *Intro to QAOA* (2026).

URL: *https://pennylane.ai/qml/demos/tutorial*$_q aoa_i ntro$

[7] Ivezic, M. (2024), 'The quantum approximate optimization algorithm (qaoa) — a beginner's guide'.

URL: *https://postquantum.com/quantum-computing/quantum-approximate-optimization-algorithm-qaoa/beyond-toy-problems-applications-of-qaoa-in-the-wild*

[8] *Portfolio Optimization* (2026).

URL: *https://qiskit-community.github.io/qiskit-finance/tutorials/01*$_p ortfolio_o ptimization.html$

[9] *QAOA for MaxCut* (2026).

URL: *https://pennylane.ai/qml/demos/tutorial*$_q aoa_m axcut$

[10] *Quantum Approximate Optimization Algorithm* (2026).

URL: *https://quantum.cloud.ibm.com/docs/en/tutorials/quantum-approximate-optimization-algorithm*

[11] *Run Your First Circuit on Hardware* (2026).

URL: *https://quantum.cloud.ibm.com/docs/en/guides/hello-world*

[12] Xan (2026), *PennyLane Documentation.*

URL: *https://docs.pennylane.ai/en/stable/*